SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	N. Sriraghavi
Register Number	1924260281
Course Code & Name	CSA02 — C Programming
Slot	D
Assignment / Task Title	Assignment-3, Q7: Diagnose, Redesign and Validate
GitHub Repository URL	https://github.com/<your-username>/<your-repo>
Submission Date	01-11-2026

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

TASK 7 — Diagnose, Redesign and Validate (Prime Testing by Trial Division)

Task / Problem Statement

From the programs written for Q1–Q6, the prime-number checker (Q5/Q6 loop design) was chosen. It tests whether n is prime by dividing n by every integer from 2 to n-1 and is run for every n from 1 to N. The task is to diagnose why it becomes slow as N grows, set a measurable objective, redesign it using a property of the problem (not just a different loop), predict the gain, and validate with operation counts.

Aim

To reduce the number of division (modulo) operations in prime testing while producing exactly the same results, and to evaluate the redesign critically using measured counts.

Concept / Theory

A number n is prime if it has no divisor other than 1 and itself. Two properties can be exploited: (1) if n has a divisor larger than sqrt(n), it must also have a matching divisor smaller than sqrt(n), so testing divisors above sqrt(n) is redundant; (2) every even number above 2 is divisible by 2, so once n is shown odd, even divisors can never divide it and need not be tried. The primary evidence is the count of divisions, because each n % i is the dominant operation inside the loop.

A. Diagnose the Bottleneck (2 marks)

Almost all work is done inside the inner for-loop, in the statement **n % i**. For a prime n the loop never exits early, so it performs n-2 divisions; for a composite n it stops at the smallest factor. Since primes are the expensive case and the work per prime grows linearly, total divisions grow roughly like $N^2/\ln N$.

N	Primes found	Divisions (original)
100	25	1,133
1000	168	78,022
10000	1229	5,775,223
20000	2262	21,267,570

Evidence: going from N = 1,000 to N = 10,000 (10× the input) raised divisions from 78,022 to 5,775,223 (about 74×), confirming super-linear growth. Work is also unnecessarily repeated: for n = 9973 (prime) the original performs 9,971 divisions although no divisor above sqrt(9973) ~ 100 can ever succeed, and it tries all even divisors even though n is odd.

B. Define the Design Objective (2 marks)

Objective: for N = 10,000, reduce the total number of divisions by at least 90% compared with the original (from 5,775,223 to below 577,522), while the list of primes found stays identical (1,229 primes).

C. Design the Revised Solution (4 marks)

What changed: (i) numbers below 2 are rejected, 2 is accepted directly, and every other even number is rejected with a single division; (ii) for odd n the loop starts at 3 and increases by 2, so even divisors are skipped; (iii) the loop condition is i*i <= n, so it stops at sqrt(n). **Property exploited:** divisors occur in pairs (d, n/d), and evenness. **Extra memory:** none — only the same integer variables. **New trade-off:** the logic is slightly harder to read and verify, and i*i can overflow a 32-bit int for n above 46,340 (use long long or i <= n/i for larger ranges).

```
function isPrime(n):
    if n < 2:            return false
    if n == 2:           return true
    if n is even:        return false          # 1 division
    for i = 3; i*i <= n; i = i + 2:
        count division
        if n mod i == 0: return false
    return true
```

D. Predict Before Implementing (1 mark)

Prediction: a prime near n = 10,000 needs about n-2 ~ 10,000 divisions originally but only about (sqrt(n)-1)/2 ~ 50 after the redesign, i.e. a per-prime reduction of roughly 200×. For the whole range 1..10,000 composites are cheap in both versions, so I expect the total to fall by **more than 90% (about 50–100× fewer divisions)**, and the saving to **grow** as N increases because sqrt(n) grows much slower than n.

Procedure / Algorithm

1. Write the original program with a global division counter incremented before every n % i.
2. Run it for N = 100, 1000, 10000 and 20000 and record primes found and divisions.
3. Write the redesigned isPrime (skip evens, start at 3 step 2, stop when i*i > n) with the same counter.
4. Run it for the same N values and record the same quantities.
5. Check both versions agree for every n from 0 to 20000 and compare counts.

Program / Source Code

Original program (original.c)

```
#include <stdio.h>

/* ORIGINAL: trial division by every i from 2 to n-1 */
long divisions = 0;

int isPrimeOriginal(int n) {
    if (n < 2) return 0;
    for (int i = 2; i < n; i++) {
        divisions++;                /* count each modulo operation */
        if (n % i == 0) return 0;
    }
    return 1;
}

int main(void) {
    int limits[] = {100, 1000, 10000, 20000};
    printf("ORIGINAL (divide by 2..n-1)\n");
    printf("%-8s %-8s %-14s\n", "N", "Primes", "Divisions");
    for (int k = 0; k < 4; k++) {
        int count = 0;
        divisions = 0;
        for (int n = 1; n <= limits[k]; n++)
            if (isPrimeOriginal(n)) count++;
        printf("%-8d %-8d %-14ld\n", limits[k], count, divisions);
    }
    return 0;
}
```

Redesigned program (redesigned.c)

```
#include <stdio.h>

/* REDESIGN: handle 2 separately, test only odd divisors, stop at sqrt(n) */
long divisions = 0;

int isPrimeRedesigned(int n) {
    if (n < 2) return 0;
    if (n == 2) return 1;
    if (n % 2 == 0) { divisions++; return 0; }   /* one division removes all evens */
    for (int i = 3; i * i <= n; i += 2) {
        divisions++;
        if (n % i == 0) return 0;
    }
    return 1;
}

int main(void) {
    int limits[] = {100, 1000, 10000, 20000};
    printf("REDESIGNED (odd divisors up to sqrt(n))\n");
    printf("%-8s %-8s %-14s\n", "N", "Primes", "Divisions");
    for (int k = 0; k < 4; k++) {
        int count = 0;
        divisions = 0;
        for (int n = 1; n <= limits[k]; n++)
            if (isPrimeRedesigned(n)) count++;
        printf("%-8d %-8d %-14ld\n", limits[k], count, divisions);
    }
    return 0;
}
```

Output Screenshot

Original program output (raw counts):

```
ORIGINAL (divide by 2..n-1)
N        Primes   Divisions
100      25       1133
1000     168      78022
10000    1229     5775223
20000    2262     21267570
```

Redesigned program output (same N values):

```
REDESIGNED (odd divisors up to sqrt(n))
N        Primes   Divisions
100      25       136
1000     168      2850
10000    1229     60957
20000    2262     155071
```

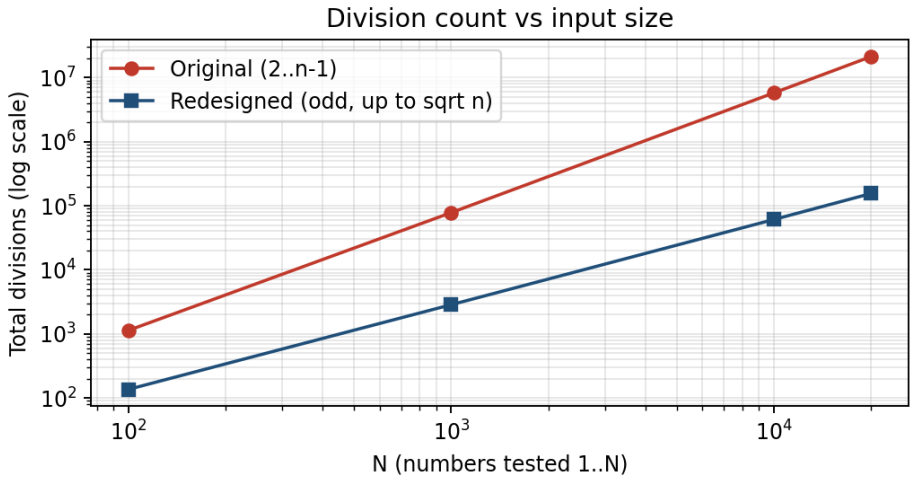
Single-number checks and agreement test over 0..20000:

```
7 prime? orig=1 new=1 divs orig=5 new=0
97 prime? orig=1 new=1 divs orig=95 new=4
9973 prime? orig=1 new=1 divs orig=9971 new=49
10007 prime? orig=1 new=1 divs orig=10005 new=49
10000 prime? orig=0 new=0 divs orig=1 new=1
10001 prime? orig=0 new=0 divs orig=72 new=36
mismatches 0..20000: 0
```

(Replace these images with your own screenshots from your compiler window if your lab requires it.)

E. Implement and Validate the Redesign (2 marks)

N	Primes (both)	Original divisions	Redesigned divisions	Reduction	Factor
100	25	1,133	136	88.00%	8.3x
1000	168	78,022	2,850	96.35%	27.4x
10000	1229	5,775,223	60,957	98.94%	94.7x
20000	2262	21,267,570	155,071	99.27%	137.1x



Both versions found the same number of primes for every N, and the agreement test reported **0 mismatches for n = 0 to 20,000**, so the redesign is behaviour-preserving.

F. Defend the Final Design (2 marks)

Objective achieved? Yes. At N = 10,000 divisions fell from 5,775,223 to 60,957, a reduction of 98.94% (94.7× fewer), well above the 90% target, and the prediction of 50–100× was met. **Size of improvement:** it grows with input — 8.3× at N = 100, 27.4× at N = 1,000, 94.7× at N = 10,000 and 137.1× at N = 20,000 — because the cost per prime changed from about n to about sqrt(n)/2. **New costs:** more complex logic, a special case for 2, and the i*i overflow risk for large n; no extra memory. **Where the original is still preferable:** for very small N (e.g. N = 100 only 997 divisions are saved), simplicity and ease of checking matter more than speed. If the real need is all primes up to N, a sieve of Eratosthenes would do far fewer operations but needs an N-sized array (extra memory), so trial division with sqrt(n) remains better when memory is tight or only single numbers are tested.

Analysis / Discussion

The counts show the original grows roughly quadratically (about 74× more work for 10× input) while the redesign grows about $N^{1.5}$ (about 21× more work for 10× input between N = 1,000 and 10,000), which is why the gap widens. For a single prime such as 10,007 the original needs 10,005 divisions and the redesign 49. Composite numbers like 10,000 cost one division in both versions, so the saving comes almost entirely from primes and odd composites with large smallest factors.

Result

The redesign met the objective with identical results and a 98.94% reduction in divisions at N = 10,000. I learned that a diagnosis based on operation counts, and a design based on a mathematical property of the problem, gives far larger gains than rewriting the loop in another form.

Test Cases / Validation Table

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1	n = 7 (small prime)	Prime	Prime (orig 5 div, new 0)	Pass
2	n = 97 (prime)	Prime	Prime (orig 95 div, new 4)	Pass
3	n = 10007 (large prime)	Prime	Prime (orig 10,005 div, new 49)	Pass
4	n = 10000 (even composite)	Not prime	Not prime (1 div each)	Pass
5	n = 10001 (odd composite, 73 x 137)	Not prime	Not prime (orig 72, new 36)	Pass
6	All n from 0 to 20000	Both versions agree	0 mismatches	Pass

Final Submission Checklist

[ ] Student and course details completed

[ ] Task statement, aim, theory, and algorithm included

[ ] Program code is readable and runs successfully

[ ] Output screenshot included

[ ] Analysis and result written in own words

[ ] Test cases completed where applicable

[ ] GitHub repository link added if required